

State-Driven Retrieval and Learned Re-Ranking for Conversational Music Recommendation

Nidhin Pattaniyil
npatta01@gmail.com
Independent Researcher
USA

Semih Yagli
semihyagli1@gmail.com
Independent Researcher
USA

Tanwir Zaman
zaman.tanwir@gmail.com
Independent Researcher
USA

Abstract

We describe team npatta01’s submission to the RecSys Challenge 2026 conversational music recommendation task: given a multi-turn dialog, retrieve 20 tracks from a 47,071-track catalog and generate a natural-language response. The pipeline extracts a typed conversation state with an LLM, gathers candidates from eleven retrieval branches over a unified track index, re-ranks the pooled candidates with a LambdaMART model whose dominant feature is a fine-tuned conversational bi-encoder cosine, and generates a state-conditioned response. On the final Blind-B leaderboard the submission scored 0.3811 composite (nDCG@20 0.2537, LLM-judge 3.30), ranking 29th of 39 teams. We describe the system, then examine the result: our development estimates were computed in-sample and overstated performance, while leakage-safe out-of-fold estimates (nDCG@20 0.197–0.203) were far closer to the blind outcome; we disagreed with many of the single-ground-truth training labels — most often when a label repeats the just-played artist against an explicit request for someone else — and re-judged all of them with LLM judges; and failure cases from the submitted run show extracted constraints the pipeline could not enforce. Code, models, and a full reproduction bundle are publicly released.

CCS Concepts

• **Information systems** → **Recommender systems**; *Retrieval models and ranking*; *Music retrieval*.

Keywords

conversational recommender systems, music recommendation, learning to rank, reciprocal rank fusion, label noise, RecSys Challenge

1 Introduction

The Music-CRS challenge [1, 2] asks systems to act as a conversational music recommender. Each session is a multi-turn dialog between a listener and an assistant; at every assistant turn the system must (i) retrieve a ranked list of 20 tracks from a shared 47,071-track catalog and (ii) write a short natural-language response presenting its top recommendation. Training data comprises 15,199 sessions built on the TalkPlayData 2 collection [3], with per-track metadata (title, artist, album, up to 37 genre/mood tags, popularity, release date) and precomputed embeddings (text, audio, cover art, collaborative filtering). Evaluation is on two 80-session blind splits of final turns; Blind-B additionally removes the conversation-goal annotations present in training data. Submissions are scored with the organizers’ composite metric:

$$0.50 \cdot \text{nDCG@20} + 0.10 \cdot \text{cat. div.} + 0.10 \cdot \text{lex. div.} + 0.30 \cdot \frac{\text{judge}-1}{4},$$

where “judge” is a 1–5 rating of the generated response from the organizers’ LLM-based evaluator. In addition to the training sessions, a 1,000-session development split with public per-turn ground truth (one target track per turn) supports offline iteration.

Our system is a retrieve-then-rerank pipeline conditioned on an explicitly compiled conversation state. The deployed reranker is *goal-free*: it uses none of the conversation-goal annotations that Blind-B withholds. This paper documents the pipeline (Section 2, Figure 1) and its results (Section 3), then analyzes the errors behind the final score (Section 4): in-sample development evaluation (Section 4.1); training labels we disagreed with and re-judged (Section 4.2); and the constraint-enforcement gaps behind concrete failure cases (Section 4.3). The reranker’s dominant feature is a fine-tuned conversational bi-encoder cosine (59% of total split gain), which displaced the artist-copy shortcut the model otherwise learned from the labels. Code, models, and the re-judged labels are released [16].

Related work in brief: the design follows the retrieve-then-rerank pattern standard in large-catalog recommendation, with reciprocal-rank fusion [4] for multi-source pooling, LambdaMART [5, 6] for learned ranking, and LLM-based dialog state tracking.

2 Approach

2.1 State extraction

At each turn we prompt an LLM (deepseek-v4-flash [14]) with the full conversation so far (all prior turns plus the user profile) to emit a schema-constrained conversation state. Its eleven fields cover the current request and intent mode, track feedback and pinned references, mentioned entities, hard filters and explicit rejections, process constraints, routing flags, lyrical theme, and a release-year window; Figure 2 shows the schema populated on a real turn. A fuzzy resolver grounds surface names to catalog IDs so downstream stages operate on identifiers. Cheaper extractors under-filled optional fields (rejections, facets), so all final runs use deepseek-v4-flash; extraction is cached per turn.

2.2 Candidate retrieval

All per-track information lives in a **single LanceDB collection** [15]: metadata fields, a full-text index, and every embedding column — three Qwen3-Embedding text views (metadata, attributes, lyrics; 1024-d), LAION-CLAP audio (512-d) [8], SigLIP-2 cover art (768-d) [9], BPR collaborative-filtering vectors (128-d) [10], and our bi-encoder document vectors (2560-d, Section 2.4). Every retrieval branch is a differently-shaped query against the same table — in principle much of the retrieval step could be expressed as a single composed query (a gap Section 4.3 returns to).

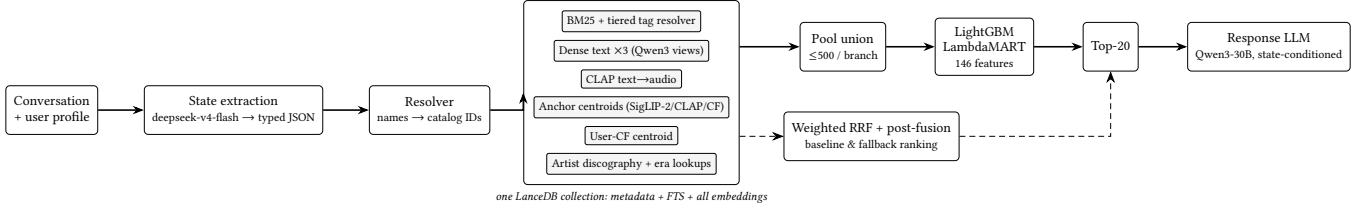


Figure 1: System overview. The conversation is compiled into a typed state; eleven retrieval branches query one LanceDB collection; the LambdaMART reranker scores the branch-pool union and emits the top-20; a response LLM presents the top track. Weighted RRF over the same branches provides the baseline and fallback ranking (dashed).

User (turn 3): "...subdued, female singer, a sense of place in the lyrics, stark almost a cappella delivery...not it either." Extracted state (abridged): request_type: hidden_target anchor artist: Neko Case (must_use) rejected tracks: "Calling Cards", "Man" facets: mood=subdued; lyrical_theme=sense of place; sonic=stark almost a cappella Top-1: Neko Case — "Bracing For Sunday" Submitted response: "You're looking for something subdued with a strong sense of place and a stark, almost a cappella delivery—and <i>Bracing For Sunday</i> fits that mood perfectly..."
--

Figure 2: A real Blind-B turn (session 024a2738. . .): extracted state and submitted output for a hidden-target request.

Table 1: The eleven retrieval branches (all queries against the one collection).

Branch	Space	Query
BM25 (fielded)	lexical	state text + resolved tags
Dense text ×3	Qwen3 views	state-built strings
CLAP text→audio	audio	sonic description
Anchor centroids ×3	SigLIP-2/CLAP/CF	liked-track centroids [†]
User-CF centroid	CF-BPR	the user's own vector
Lookups ×2	catalog	discography; era popularity

[†] Gated by intent mode: a pivot suppresses anchor centroids.

Table 1 lists the eleven branches that produce candidate pools each turn. In the BM25 branch [11], free-text mood and genre phrases first pass through a **tiered tag resolver**: exact, alias, and substring matching against the catalog tag vocabulary, with an embedding nearest-neighbor fallback (cosine ≥ 0.6 , top-3). Weighted reciprocal-rank fusion [4] ($k=60$, top-1000) plus post-fusion multiplicative adjustments (rejected artists/tracks dropped, played tracks dropped, exact-membership tag promotes/demotes, year-range decay) yields an explicit ranking that serves as **baseline and fallback**.

2.3 Candidate re-ranking

The submitted system replaces the fused order: a LightGBM LambdaMART [5, 6] model scores the **union of the branch pools** (each truncated to its top 500) and emits the final top-20. It sees 146 features per (turn, track) pair: per-branch rank/score/margin/hit features, pool-normalized z-scores and percentiles, content and behavioral cosines, session/artist-affinity counters, tag and lexical overlap, popularity, extracted-state categoricals, and a constraint sidecar. Table 2 groups them by family with each family's share of

Table 2: Reranker feature families (share of total gain, deployed model).

Family	#	Gain	Examples
Bi-encoder cosine	1	59.0%	b1_cos
Per-branch rank/score	66	11.5%	BM25 rank, dense margins
Session/artist affinity	18	10.7%	same-artist, played count
Other similarity cosines	17	8.9%	CLAP/SigLIP centroids
Popularity	7	5.0%	pop. percentile
State/intent	14	2.2%	request type, intent mode
Tag/lexical overlap	10	2.0%	IDF tag overlap
Constraints/rejections	13	0.7%	rejected-artist flag

total split gain in the deployed model; one feature — the bi-encoder cosine b1_cos — carries 59.0% of all gain. The RRF rank itself is *excluded*: the reranker sees raw branch evidence, not the fused opinion. After scoring, two rule-based guards apply: an exact-track pin when the user names a specific track, and a final-artist constraint check.

The deployed model is *goal-free*, trained without the conversation-goal annotations that Blind-B removes. Its training data is the development split only — the same 8,000 turns used for offline evaluation (~20.6M candidate rows): the state extraction that per-turn retrieval tracing depends on runs through a paid LLM API, and we never built features over the ~121.6k-turn training split; Section 4.1 discusses the evaluation consequence of this reuse. Training uses the per-turn ground-truth track as a binary label (LambdaRank objective optimizing nDCG@20; learning rate 0.025; 127 leaves; truncation 200; 5-fold user-grouped cross-validation). Because we disagreed with many labels (Section 4.2), labels are down-weighted when the *next* turn contradicts them: $\times 0.3$ if goal progress says the recommendation did not move toward the goal, $\times 0.3$ if the user rejects the ground-truth track, $\times 0.6$ if they reject its artist.

2.4 The bi-encoder feature b1_cos

The reranker's dominant feature comes from a two-tower bi-encoder fine-tuned from Qwen3-Embedding-4B [7]. The query tower renders the conversation compactly behind a retrieval instruction prefix and the document tower renders a track card (both formats in Figure 3); the card's one-line LLM-written "known for" artist description imports static world knowledge from larger models into the embedding space. One shared encoder E_θ serves both towers; with $v_x = E_\theta(x)/\|E_\theta(x)\|_2$ (last-token pooling, 2560-d), the feature

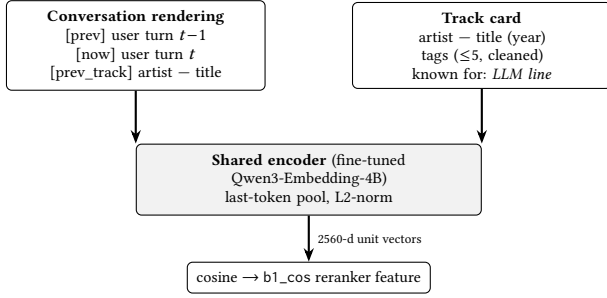


Figure 3: The b1 bi-encoder: one shared fine-tuned encoder embeds the conversation and the track card; their cosine is the reranker’s dominant feature.

is the cosine $b1_cos(q, d) = v_q^\top v_d$. Training minimizes the in-batch contrastive (MNRL [13]) loss over batches of positive pairs (q_i, d_i^+) with four mined hard negatives per positive,

$$\mathcal{L} = -\frac{1}{B} \sum_i \log \frac{\exp(v_{q_i}^\top v_{d_i^+} / \tau)}{\sum_{d \in \mathcal{D}_i} \exp(v_{q_i}^\top v_d / \tau)}, \quad \tau = \frac{1}{20},$$

where $\mathcal{D}_i$ contains the in-batch documents plus the hard negatives. Positives are conservative: the 53,885 (turn, track) pairs whose next-turn goal-progress annotation says the recommendation *moved the session forward*. Document vectors for all 47,071 tracks are precomputed into the collection; the conversation vector is encoded live with caching.

We deploy the bi-encoder as a reranker feature only: as a retrieval branch it added little coverage, but its cosine became the top signal (Table 2) — and a *generalizing replacement* for the artist-copy shortcut (Section 4.2): in leakage-safe out-of-fold ablations, removing the artist-copy shortcut features reduces nDCG@20 by 0.0070, while adding b1_cos more than recovers the loss (+0.0091) with the shortcut absent.

2.5 Response generation

The response generator is Qwen3-30B-A3B-Instruct-2507 [12] at temperature 0: a single pass over the top-1 track rendered as a delimited XML block (≤ 10 tags) plus the *latest* extracted state. The deployed template’s full style instruction is:

“Write 1–2 concise sentences about only the selected track. Prioritize the latest user request and extracted state over older conversation history. If the track is reasonably aligned, explain the fit with one specific supported reason. If it clearly conflicts with an explicit avoid/new-artist constraint, do not oversell it or blame the system; briefly frame the limitation and the closest supported reason.”

Over a frozen Blind-A retrieval output, a sweep of eight templates (varying state conditioning, history depth, item rendering, and concision) moved our offline judge score from 3.95 to 4.70 without changing the recommendations. There is no multi-draft sampling, selection, checking, or repair pass.

Table 3: Development split. The in-sample row replays turns the deployed (full-data) model saw during training; the out-of-fold rows are leakage-safe cross-validation estimates.

System (how measured)	nDCG@20
Weighted RRF fusion (no training)	0.1492
LambdaMART, out-of-fold CV	0.1970
+ b1_cos (deployed feature set)	0.2032
LambdaMART, evaluated in-sample (train replay)	0.3844

Official Blind-B nDCG@20: 0.2537 — above the out-of-fold estimates and far below the in-sample replay.

Table 4: Official CodaBench results; the composite formula is defined in Section 1. The Blind-A rank counts all development-phase submissions.

Split	nDCG	Cat.	Lex.	Judge	Comp.	Rank
Blind-A (dev)	0.4380	0.0313	0.7670	4.20	0.5389	63/181
Blind-B (final)	0.2537	0.0315	0.7862	3.30	0.3811	29/39

Code, configs, trained models, cached extraction states, and a frozen-LLM-cache reproduction bundle that replays our submissions without API credentials are public.¹

3 Results

The development split (1,000 sessions $\times$ 8 turns) has one ground-truth track per turn. Table 3 reports development-split numbers, separating leakage-safe estimates from in-sample ones; Table 4 reports the official leaderboard results.

Learned re-ranking improves over static fusion by the leakage-safe estimate (0.149 $\rightarrow$ 0.197–0.203); Section 4.1 discusses the in-sample row.

4 Error Analysis

4.1 In-sample development evaluation

Per-turn state extraction and retrieval tracing made held-out evaluation sets expensive, so the development split served both training and evaluation. The deployed reranker was fit on all of its feature data (cross-validation informed early stopping; a full-data model shipped), making the headline development estimates (0.38–0.46 across reranker versions) in-sample — and those numbers drove selection. The leakage-safe out-of-fold estimates (0.197–0.203) predicted the blind outcome (0.2537) far better; the in-sample replay (0.384) had overstated it. We regard this as a measurement error rather than a demonstrated cause of the final ranking.

4.2 Label disagreement and relabeling

While building training data we found many turns where we disagreed with the label: the labeled target did not seem to fit the user’s request. The per-turn ground truth is the track actually played next in the listener’s real session, not an editorial judgment of the request, so some mismatch is expected. In a manual sample we judged

¹<https://github.com/npatta01/music-crs-2026>; models and data: <https://huggingface.co/datasets/Npatta01/music-crs-repro-2026>.

User: "...something with a similar chill, electronic vibe, but from a different artist? " (just played: <i>Bonobo</i>)	
Ground truth: Bonobo — "Jets"	annotation: MOVES toward goal
User (next turn): "‘Jets’ is cool, but I was actually hoping for something from a different artist this time... "	
Ground truth: Bonobo — "Pieces"	annotation: MOVES toward goal

Figure 4: The anchoring pattern (real session): two consecutive explicit different-artist requests, yet the label stays on Bonobo and the goal-progress annotation records both turns as moving toward the goal.

Table 5: Label-free audit of the submitted Blind-B run: where the flagged turns were lost.

Diagnosis	Turns
Better candidate in pool, ordered below top-20	31
Over-aggressive filtering	8
State-extraction miss	3
Entity-resolution miss	1

62% of labeled tracks a clear fit to the request, 26% a loose fit, and 11% no fit. The pattern we disagreed with most often is what we call **anchoring**: the label stays with the just-played artist even against an explicit request for someone else (Figure 4).

To measure the disagreement at scale, two inexpensive LLM judges scored every training and development turn (113,393 in total) on two axes — whether the user asked for a different artist, and whether the track fits the request — with a stronger arbiter re-judging every conflict without access to the goal-progress annotation. Our judges disagreed with 57.4% of the labels; 18,222 of the disagreements are anchoring cases, and in 5,880 of those the goal-progress annotation additionally records the anchored track as moving toward the goal — pairs we treat as poisoned positives for any goal-progress-based training scheme, including our bi-encoder’s. A second pattern, in which the user rejects the labeled track on the next turn, motivated the $\times 0.3$ down-weighting of Section 2.3. Training on the labels as-is rewards copying the last artist; our ablations show the reranker exploiting exactly that shortcut until the bi-encoder cosine displaced it (Section 2.4). Our judges, rather than the labels, may of course be the misjudging party, so we release the full relabeling for independent audit; the submitted models were *not* trained on it.

4.3 Failure cases

A separate label-free audit — a single LLM judge over the 80 submitted Blind-B turns, unlike the two-judge pipeline of Section 4.2 — rated 68% of them weak-or-bad fits; Table 5 breaks a subset of the flagged turns down by failure diagnosis (hidden-label frequencies remain unknown). The largest bucket — a better candidate present in the pool but ranked below the top-20 — is the ranking gap the closing paragraph returns to. The cases below, from the submitted predictions, illustrate the smaller buckets and adjacent gaps the table’s categories only partly capture:

- **Anchoring at serving time.** “Another alt-country song... but **from a different artist than Ryan Adams**” → our top-1

was *Ryan Adams & The Cardinals*: the rejection resolved to the solo “Ryan Adams” catalog ID, and the band-variant ID passed the hard filter. **Gap**: rejections are enforced by exact catalog ID, while artist identity is fragmented across variant and duplicate IDs; a name-level veto would have caught this. The bucket is small (Table 5), but it is the serving-time face of the Section 4.2 anchoring pattern.

- **Impossible exact request.** “Play ‘Watercolors’ by Pat Metheny” — the track does not exist in the 47,071-track catalog; we returned Pat Metheny’s “Alfie” without saying so. **Gap**: the failed catalog resolution was never passed to the response generator, and without external data the system cannot distinguish an out-of-catalog track from a misremembered one; the single-pass response therefore could not flag unavailability — costly, since the response term carries 30% of the composite (our judge score: 3.30).
- **Unactionable constraint.** “Any aggressive metal track that **exactly matches 126.70 bpm and G minor**, with tempo and key stated” — the catalog has no BPM or key fields. **Gap**: the constraint was captured in state, but the catalog has no field to map it onto.

The primary limitation is that **the system does not model a sufficient portion of the user’s intent as executable constraints**. The state records what the user wants, but few fields became filters or ranked evidence — each branch consumed a slice of the state, and tag matching fell back to exact strings outside the BM25 branch — while no co-occurrence, transition, or live-reasoning lane existed to compensate. Noisy candidate pools therefore reached the reranker, which could not reliably rank the target track within the top-20 — Table 5’s largest bucket.

5 Conclusion

We documented a state-compiled retrieve-then-rerank conversational music recommender and finished 29th of 39. Three gaps in our approach account for the failures we could diagnose:

- (1) model selection relied on in-sample development replays, although out-of-fold estimates that predicted the blind score were on hand;
- (2) the extracted conversation state was rich, but too little of it became executable filters or ranking evidence, so noisy candidate pools reached the reranker;
- (3) responses were generated in a single unchecked pass that could flag neither unavailable tracks nor unmet constraints, under a judge term worth 30% of the composite.

Beyond these gaps, we disagreed with many of the training labels and re-judged all 113,393 turns with LLM judges (Section 4.2). We release the pipeline, the models, a replay bundle, and the re-judged labels.

References

- [1] NLP4MusA organizers. 2026. Music-CRS Challenge 2026. <https://nlp4musa.github.io/music-crs-challenge/>
- [2] RecSys Challenge organizers. 2026. RecSys Challenge 2026. <https://www.recsyschallenge.com/2026/>
- [3] TalkPlay / TalkPlayData 2: multimodal conversational music recommendation dataset. <https://huggingface.co/collections/talkpl-ai/talkplay-data-challenge>

- [4] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. 2009. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *SIGIR*.
- [5] Guolin Ke et al. 2017. LightGBM: a highly efficient gradient boosting decision tree. In *NeurIPS*.
- [6] Christopher J. C. Burges. 2010. From RankNet to LambdaRank to LambdaMART: an overview. MSR-TR-2010-82.
- [7] Qwen Team. 2025. Qwen3 Embedding: advancing text embedding and reranking. arXiv:2506.05176.
- [8] Yusong Wu et al. 2023. Large-scale contrastive language-audio pretraining. In *ICASSP*.
- [9] Michael Tschannen et al. 2025. SigLIP 2: multilingual vision-language encoders. arXiv:2502.14786.
- [10] Steffen Rendle et al. 2009. BPR: Bayesian personalized ranking from implicit feedback. In *UAI*.
- [11] Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in IR*.
- [12] Qwen Team. 2025. Qwen3 technical report. arXiv:2505.09388.
- [13] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: sentence embeddings using Siamese BERT-networks. In *EMNLP*.
- [14] DeepSeek. 2026. DeepSeek-V4-Flash model card (OpenRouter).
- [15] LanceDB: serverless vector database. <https://lancedb.com>
- [16] Team npatta01. 2026. music-crs-2026: code, models, and reproduction bundle. <https://github.com/npatta01/music-crs-2026>